

EXPERIMENT 2 — WATER JUG PROBLEM

AIM

To write a Python program to solve the **Water Jug Problem using Breadth-First Search (BFS)**.

ALGORITHM – WATER JUG PROBLEM

1. Start with both jugs empty: **(0, 0)**.
2. Use a queue to store possible states.
3. Mark each visited state to avoid repetition.
4. Generate possible operations:
 - Fill Jug A.
 - Fill Jug B.
 - Empty Jug A.
 - Empty Jug B.
 - Pour water from A to B.
 - Pour water from B to A.
5. Add all unvisited states to the queue.
6. Check whether either jug contains the required **2 litres**.
7. If the goal is reached, display the sequence of states.
8. Otherwise, continue until the solution is found.
9. Stop.

PYTHON PROGRAM :

```
from collections import deque
```

```
A = 4
```

```
B = 3
```

```
goal = 2
```

```
queue = deque([(0, 0), []])
```

```
visited = set()
```

```
while queue:
```

```
    (x, y), path = queue.popleft()
```

```
    if (x, y) in visited:
```

```
continue
```

```
visited.add((x, y))
```

```
if x == goal or y == goal:
```

```
    print("Solution:")
```

```
    for state in path + [(x, y)]:
```

```
        print(state)
```

```
    break
```

```
states = [
```

```
    (A, y), # Fill Jug A
```

```
    (x, B), # Fill Jug B
```

```
    (0, y), # Empty Jug A
```

```
    (x, 0), # Empty Jug B
```

```
    # Pour A into B
```

```
    (x - min(x, B-y), y + min(x, B-y)),
```

```
    # Pour B into A
```

```
    (x + min(y, A-x), y - min(y, A-x))
```

```
]
```

```
for state in states:
```

```
    if state not in visited:
```

```
        queue.append((state, path + [(x, y)]))
```

OUTPUT:

`Solution:`

`(0, 0)`

`(0, 3)`

`(3, 0)`

`(3, 3)`

`(4, 2)`

RESULT

Thus, the **Water Jug Problem** was successfully solved using **Breadth-First Search** in **Python**.